

QUESTIONS ONE

[100 MARKS]

NeoVerse: AI City Survival System (Console-Based C++ Game Engine)

Background Context

By the year 2035, urban environments will no longer be governed solely through human oversight. Instead, they will operate under the integrated administration of advanced AI Control Cores, which autonomously regulate critical municipal systems, including energy distribution, transportation networks, digital citizen management, and emergency-response infrastructures in real time.

You have been recruited as a Junior Systems Engineer at NeoVerse Labs, a global tech company responsible for maintaining the AI City Survival Engine, a prototype console-based system used to train engineers before they work on real smart cities.

The city is under constant pressure:

- ❖ NPC citizens generate **events** every second
- ❖ Resources must be managed efficiently
- ❖ Emergency alerts must be handled in the correct order
- ❖ Decisions must be optimized for performance (Big-O matters!)

Your task is to build a console-based C++ simulation system that manages the city's data, events, and agents using STL containers, algorithms, and object-oriented programming principles.

This system will demonstrate your understanding of:

- ❖ Dynamic data structures
- ❖ Algorithm efficiency
- ❖ Object-oriented design
- ❖ Real-world problem solving using C++

1. AI Agent Authentication & Access Control

(15 MARKS)

1.1 AI Engineer Login System

- Engineers must log in before accessing the city system.
- Implement an **Engineer class** with these attributes and interfaces to interact with them:
 - ❖ Engineer ID (ENG001, ENG002)
 - ❖ Username
 - ❖ Encrypted password
 - ❖ Clearance level (Low, Medium, High)

STL Usage

- ❖ Store engineers in a **vector container**
- ❖ Use <algorithm> to search for valid credentials

Big-O Focus

- ❖ Analyse login search complexity (linear vs binary search)

2. City Data Management (Dynamic Containers)

(20 MARKS)

The city constantly generates data, population stats, energy usage, traffic density, and alerts.

2.1 City Data Storage

- ❖ Use a **vector** to store daily sensor readings
- ❖ Use a **linked list** to store historical city logs (unlimited growth)

Explain:

- ❖ Why is a vector suitable for fast access
- ❖ Why is a linked list suitable for unbounded logs

2.2 Data Operations

- ❖ Insert new data
- ❖ Remove outdated data
- ❖ Traverse and display data

Big-O Analysis

- ❖ Compare insertion and traversal complexities

3. Event Processing System (Real-Time Simulation)

(25 MARKS)

Events occur constantly in the city:

- ❖ Traffic accident
- ❖ Power failure
- ❖ Network overload
- ❖ Weather alert

3.1 Event Queue (FIFO)

- ❖ Use a **queue** to manage incoming city events
- ❖ Events must be processed in the order they arrive

3.2 Emergency Override Stack (LIFO)

- ❖ Use a **stack** for emergency overrides
- ❖ The most recent emergency must be resolved first

STL Containers

- ❖ `queue<Event>`
- ❖ `stack<EmergencyEvent>`

Algorithms

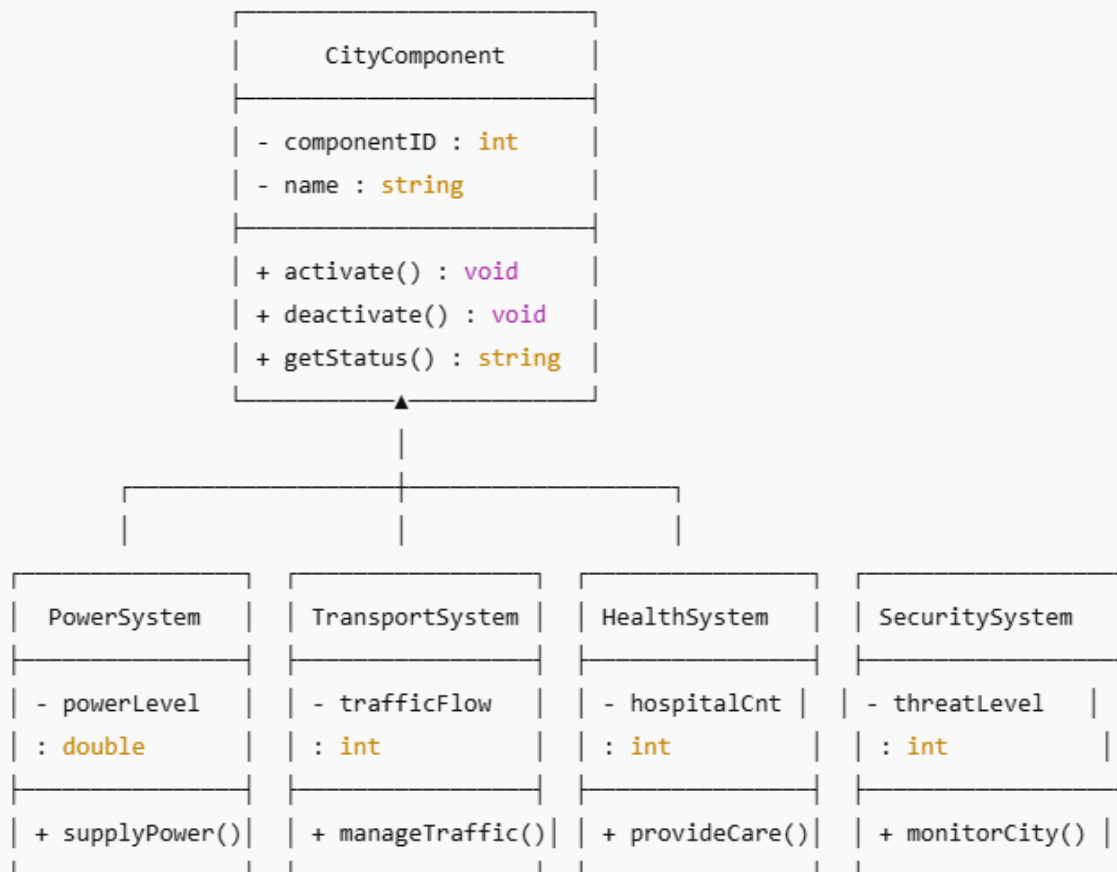
- ❖ Use `<algorithm>` to prioritise or filter events
- ❖ Explain why stack vs queue is appropriate

4. Object-Oriented City Architecture

(15 MARKS)

4.1 Class Design

Implement the following class hierarchy:



Each derived class must:

- ❖ Override a virtual function `processEvent()`
- ❖ Demonstrate polymorphism

OOP Concepts Required

- ❖ Encapsulation
- ❖ Inheritance
- ❖ Polymorphism
- ❖ Constructors & destructors

5. STL Algorithms & Performance Optimisation

(15 MARKS)

5.1 Algorithm Usage

Use STL algorithms to:

- ❖ Sort sensor data
- ❖ Find highest/lowest values
- ❖ Count critical alerts
- ❖ Search for a specific event

Required algorithms:

- ❖ sort
- ❖ find
- ❖ min_element
- ❖ max_element
- ❖ count_if

5.2 Big-O Justification

For each algorithm used:

- ❖ State the time complexity
- ❖ Justify why it was chosen

6. System Reports & Analytics

(5 MARKS)

Generate reports such as:

- ❖ Total events processed
- ❖ Most common emergency type
- ❖ Average response time
- ❖ System load summary

Use:

- ❖ Iterators
- ❖ STL containers
- ❖ Algorithm library

7. File Handling & Persistence

(10 MARKS)

7.1 Data Storage

Persist system data using files:

- ❖ engineers.dat
- ❖ events.dat
- ❖ city_logs.dat
- ❖ config.txt
- ❖ Requirements:
- ❖ Save and load system state
- ❖ Demonstrate file I/O
- ❖ Export logs to text or CSV

Submission Requirements

- All .cpp and .h files
- Data files used for testing
- README / User Guide explaining:
 - How to run the simulation
 - Sample login credentials
 - Explanation of containers used
 - Big-O decisions
- Please ensure that your assignment is submitted as a single zipped folder containing your complete project files.

NB: The total file size must not exceed 50MB.

END OF ASSIGNMENT!!!